

C++ Course

Materials for Students

Paweł Kisielewicz pkisiel@gmail.com

Krakow, 18 June 2026

Contents

1	Introduction	3
2	English Version Plan	4
2.1	Goal	4
2.2	Learning Path	4
2.3	Translation Batches	4
2.3.1	Batch 1	4
2.3.2	Batch 2	4
2.3.3	Batch 3	4
2.3.4	Batch 4	5
2.3.5	Batch 5	5
2.3.6	Batch 6	5
2.4	Editorial Rules	5
2.5	YouTube Support	5
2.6	First Downloadable English Draft	5
3	00 - Start	7
3.1	Learning Goals	7
3.2	What You Need	7
3.3	First Compiler Check	7
3.4	First Program	7
3.5	Editor	8
3.6	Git Basics	8
3.7	CMake in One Sentence	8
3.8	Student Checklist	8
3.9	Next Step	8
4	01 - C++ Basics	9
4.1	Learning Goals	9
4.2	Minimal Program Structure	9
4.3	Output	9
4.4	Input	10
4.5	Basic Types	10
4.6	Simple Calculation	10
4.7	Formatting Output	11
4.8	Common Mistakes	11
4.9	Practice Tasks	11
4.10	Student Checklist	11
4.11	Next Step	12
5	02 - Control Flow and Loops	13
5.1	Learning Goals	13
5.2	Conditions	13
5.3	Multiple Conditions	14
5.4	<code>switch</code>	14
5.5	<code>for</code> Loop	15

5.6	<code>while</code> Loop	15
5.7	<code>do while</code> Loop	15
5.8	<code>break</code> and <code>continue</code>	16
5.9	Common Mistakes	16
5.10	Bridge Exercise: Minimum, Maximum, and Average	16
	5.10.1 Requirements	16
	5.10.2 Suggested Work Plan	17
	5.10.3 Check Scenarios	17
5.11	Practice Tasks	17
5.12	Student Checklist	17
6	03 - Functions, Arrays, and Strings	18
6.1	Learning Goals	18
6.2	Why Functions Matter	18
6.3	Declaration and Definition	18
6.4	Parameters and Return Values	19
6.5	Local Variables and Scope	19
6.6	One-Dimensional Arrays	19
6.7	Array Minimum and Maximum	19
6.8	Two-Dimensional Arrays	20
6.9	Strings	20
6.10	Reading a Full Line	20
6.11	Bridge Exercise: GADERYPOLUKI Cipher	21
6.12	Required Function	21
6.13	Suggested Work Plan	21
6.14	Check Scenarios	21
6.15	Extra Tasks	22
6.16	Common Mistakes	22
6.17	Practice Tasks	22
6.18	Student Checklist	22

Chapter 1

Introduction

- Author: Paweł Kisielewicz <pkisiel@gmail.com>
- Place and date: Krakow, 18 June 2026
- Version: 0.4.0-en

This ebook is the first English draft based on the Polish C++ learning materials. The Polish version remains the source of truth for now. The English version starts with the course structure, learning path, and translation plan.

The full course is organized from environment setup and C++ basics to functions, arrays, strings, files, exceptions, object-oriented programming, STL, build systems, testing, and final projects.

Chapter 2

English Version Plan

2.1 Goal

The English edition should become a downloadable version of the C++ course for students who want to learn from the same material as the Polish course.

The first release should focus on readability and navigation. The code examples can stay close to the Polish repository at the beginning, while explanations, chapter introductions, exercises, and project descriptions are translated in batches.

2.2 Learning Path

1. `00-start` - development environment, compiler, editor, Git, and CMake.
2. `01-basics` - first program, input/output, types, variables, and operators.
3. `02-control-flow-loops` - conditions, `switch`, loops, `break`, `continue`.
4. `03-functions-arrays-strings` - functions, arrays, matrices, and strings.
5. `04-pointers-references-memory` - addresses, pointers, references, and safe memory habits.
6. `05-files-exceptions` - text files, validation, and exceptions.
7. `06-oop` - classes, objects, constructors, encapsulation, inheritance, and operators.
8. `07-stl-data-structures` - STL containers, algorithms, maps, stacks, queues, and lists.
9. `08-project-build-tests` - multi-file projects, build scripts, tests, and CI.
10. `09-modern-cpp` - selected modern C++ features, lambdas, move semantics, namespaces, and threads.
11. `10-projects` - final project topics, grading checklists, and project planning.

2.3 Translation Batches

2.3.1 Batch 1

- Main repository introduction - draft added.
- Ebook front matter - draft added.
- `00-start` - first English chapter added.
- `01-basics` - first English chapter added.

2.3.2 Batch 2

- `02-control-flow-loops` - first English chapter added.
- Bridge exercise: minimum, maximum, and average - included in the chapter.

2.3.3 Batch 3

- `03-functions-arrays-strings` - first English chapter added.
- Bridge exercise: GADERYPOLUKI cipher - included in the chapter.

2.3.4 Batch 4

- 04-pointers-references-memory.
- 05-files-exceptions.
- 06-oop.
- Bridge exercise: figures and polymorphism.

2.3.5 Batch 5

- 07-stl-data-structures.
- 08-project-build-tests.
- Bridge exercise: RPN calculator.

2.3.6 Batch 6

- 09-modern-cpp.
- 10-projects.
- Project checklist and grading card.

2.4 Editorial Rules

- Keep the English text simple and practical.
- Do not translate code identifiers unless the example itself is rewritten.
- Prefer short paragraphs and clear section titles.
- Keep exercises close to the Polish version, but adjust wording for an English-speaking student.
- Do not include archive materials in the ebook.
- Keep the Polish and English ebook versions buildable by the same script.

2.5 YouTube Support

The first bilingual video format should be short quizzes:

- Polish question first.
- English question second.
- One C++ concept per video.
- Link to the repository and both ebook downloads in the description.

Example:

C++ Quiz #1

What will this program print?

```
int x = 5;
std::cout << x++ << " " << x;
```

- A) 5 5
- B) 5 6
- C) 6 6

Answer: B) 5 6, because postfix `x++` returns the current value first and increments the variable afterwards.

2.6 First Downloadable English Draft

The first downloadable English file should be marked clearly as a draft:

- title: C++ Course
- version: 0.1.0-en
- scope: plan, learning path, and translation roadmap
- output files:

- `ebook/download/cpp-course-en.pdf`
- `ebook/download/cpp-course-en.epub`

Chapter 3

00 - Start

This chapter prepares the technical environment for learning C++. It is meant for students who are starting with a compiler, code editor, terminal, Git, and simple project structure.

3.1 Learning Goals

After this chapter you should be able to:

- explain what a C++ compiler does,
- compile and run a simple `.cpp` file,
- use a terminal for basic commands,
- open and edit source files in Visual Studio Code or another editor,
- understand why Git is useful during programming exercises,
- understand the role of `CMakeLists.txt` in larger projects,
- move safely to the first C++ programming chapter.

3.2 What You Need

You need:

- a C++ compiler with C++17 support,
- a text editor or IDE,
- a terminal,
- Git,
- access to this repository.

On macOS and Linux, the compiler is often available as `c++`, `clang++`, or `g++`. On Windows, students usually use MSYS2, Visual Studio Build Tools, or an IDE configured by the teacher.

3.3 First Compiler Check

Open a terminal and run:

```
c++ --version
```

If the command prints compiler information, you can continue. If it says that the command is missing, install a compiler before moving forward.

3.4 First Program

Create a file named `hello.cpp`:

```
#include <iostream>
```



```
int main() {  
    std::cout << "Hello, C++!" << std::endl;  
    return 0;  
}
```

Compile it:

```
c++ -std=c++17 -Wall -Wextra -pedantic hello.cpp -o hello
```

Run it:

```
./hello
```

Expected output:

```
Hello, C++!
```

3.5 Editor

Use an editor that lets you:

- open folders,
- edit `.cpp` files,
- open an integrated terminal,
- see compiler errors clearly.

Visual Studio Code is a practical default for beginners, but the course does not depend on one specific editor.

3.6 Git Basics

Git helps you save progress and review changes. For this course, the most important commands are:

```
git status  
git add file.cpp  
git commit -m "Describe the change"
```

At the beginning, treat Git as a history of your learning steps. Commit small, working changes.

3.7 CMake in One Sentence

CMake is a build configuration tool. In small exercises you can compile one file manually. In larger projects, CMake or a build script describes how many source files should be compiled together.

3.8 Student Checklist

Before going to the next chapter, make sure you can:

- open a terminal,
- check the compiler version,
- compile `hello.cpp`,
- run the generated program,
- explain what source file and executable mean,
- check Git status in a repository.

3.9 Next Step

Go to chapter 01 - Basics and write your first simple console programs.

Chapter 4

01 - C++ Basics

This chapter is the first programming block of the course. It introduces the structure of a C++ program, input and output, basic data types, variables, operators, and simple console tasks.

4.1 Learning Goals

After this chapter you should be able to:

- describe the role of an algorithm, a program, and the `main` function,
- write, compile, and run a simple C++ program,
- use `std::cout`, `std::cin`, and `std::getline`,
- choose basic data types for a simple problem,
- declare and initialize variables,
- perform simple calculations,
- format basic text output,
- solve small console exercises.

4.2 Minimal Program Structure

Most beginner programs in this course start like this:

```
#include <iostream>

int main() {
    std::cout << "Hello" << std::endl;
    return 0;
}
```

Important parts:

- `#include <iostream>` makes console input/output available,
- `int main()` is the starting point of the program,
- statements inside `{ }` are executed in order,
- `return 0;` means the program finished successfully.

4.3 Output

Use `std::cout` to print text:

```
#include <iostream>

int main() {
    std::cout << "C++ basics" << std::endl;
```

```
    std::cout << "Second line" << std::endl;
}
```

`std::endl` ends the current line.

4.4 Input

Use `std::cin` to read a value:

```
#include <iostream>
#include <string>

int main() {
    std::string name;

    std::cout << "Your name: ";
    std::cin >> name;

    std::cout << "Hello, " << name << "!" << std::endl;
}
```

Use `std::getline` when you want to read a full line with spaces:

```
#include <iostream>
#include <string>

int main() {
    std::string fullName;

    std::cout << "Full name: ";
    std::getline(std::cin, fullName);

    std::cout << "Hello, " << fullName << "!" << std::endl;
}
```

4.5 Basic Types

Common beginner types:

Type	Example use
<code>int</code>	whole numbers
<code>double</code>	decimal numbers
<code>char</code>	one character
<code>bool</code>	true or false
<code>std::string</code>	text

Example:

```
int age = 20;
double price = 19.99;
bool active = true;
std::string city = "Krakow";
```

4.6 Simple Calculation

```
#include <iostream>

int main() {
```

```

double a = 0.0;
double b = 0.0;

std::cout << "First number: ";
std::cin >> a;

std::cout << "Second number: ";
std::cin >> b;

std::cout << "Sum: " << a + b << std::endl;
std::cout << "Product: " << a * b << std::endl;
}

```

4.7 Formatting Output

For decimal precision, use `<iomanip>`:

```

#include <iomanip>
#include <iostream>

int main() {
    double value = 10.0 / 3.0;
    std::cout << std::fixed << std::setprecision(2);
    std::cout << value << std::endl;
}

```

Expected output:

3.33

4.8 Common Mistakes

- forgetting `#include <iostream>`,
- missing semicolon at the end of a statement,
- using `int` when the result should contain decimals,
- mixing `std::cin >> value` and `std::getline` without understanding how input is buffered,
- ignoring compiler warnings.

4.9 Practice Tasks

1. Print your name, city, and favorite programming topic.
2. Read two integers and print their sum, difference, and product.
3. Read a rectangle width and height, then print its area.
4. Read a temperature in Celsius and print the value in Fahrenheit.
5. Read a full name with spaces and print a greeting.

4.10 Student Checklist

Before moving to control flow and loops, make sure you can:

- write a minimal C++ program,
- compile it with `-std=c++17 -Wall -Wextra -pedantic`,
- read one value from the user,
- read one full line from the user,
- choose between `int`, `double`, `bool`, and `std::string`,
- format a decimal number with two digits after the decimal point.

4.11 Next Step

The next topic is control flow: `if`, `else`, `switch`, and loops.

Chapter 5

02 - Control Flow and Loops

This chapter teaches how a program makes decisions and repeats instructions. These tools are necessary for almost every non-trivial console program.

5.1 Learning Goals

After this chapter you should be able to:

- use `if`, `else if`, and `else`,
- use comparison and logical operators,
- use `switch` for simple menu-like choices,
- use the conditional operator `?:` in simple expressions,
- write loops with `for`, `while`, and `do while`,
- choose the right loop type for a problem,
- use `break` and `continue` in simple cases,
- solve tasks that require decisions and repetition.

5.2 Conditions

Use `if` when the program should execute code only in some situations:

```
#include <iostream>

int main() {
    int age = 0;

    std::cout << "Age: ";
    std::cin >> age;

    if (age >= 18) {
        std::cout << "Adult" << std::endl;
    } else {
        std::cout << "Minor" << std::endl;
    }
}
```

Common comparison operators:

Operator	Meaning
<code>==</code>	equal
<code>!=</code>	not equal
<code><</code>	less than
<code><=</code>	less than or equal

Operator	Meaning
>	greater than
>=	greater than or equal

Logical operators:

Operator	Meaning
&&	and
	or
!	not

5.3 Multiple Conditions

Use `else if` when there are several possible ranges or categories:

```
#include <iostream>

int main() {
    int points = 0;

    std::cout << "Points: ";
    std::cin >> points;

    if (points >= 90) {
        std::cout << "Very good" << std::endl;
    } else if (points >= 60) {
        std::cout << "Passed" << std::endl;
    } else {
        std::cout << "Failed" << std::endl;
    }
}
```

5.4 switch

Use `switch` for simple choices based on one value:

```
#include <iostream>

int main() {
    int option = 0;

    std::cout << "1. Add\n";
    std::cout << "2. Remove\n";
    std::cout << "Option: ";
    std::cin >> option;

    switch (option) {
        case 1:
            std::cout << "Add selected" << std::endl;
            break;
        case 2:
            std::cout << "Remove selected" << std::endl;
            break;
        default:
            std::cout << "Unknown option" << std::endl;
    }
}
```

```

        break;
    }
}

```

Remember `break`. Without it, execution continues into the next case.

5.5 for Loop

Use `for` when you know how many times the loop should run:

```

#include <iostream>

int main() {
    for (int i = 1; i <= 5; ++i) {
        std::cout << i << std::endl;
    }
}

```

Typical uses:

- counting from 1 to n,
- summing numbers,
- printing a table,
- iterating over an index range.

5.6 while Loop

Use `while` when the loop depends on a condition that may change during input:

```

#include <iostream>

int main() {
    int number = 0;

    while (number != 5) {
        std::cout << "Enter 5 to stop: ";
        std::cin >> number;
    }
}

```

This loop may execute zero times if the condition is false at the beginning.

5.7 do while Loop

Use `do while` when the body should run at least once:

```

#include <iostream>

int main() {
    int option = 0;

    do {
        std::cout << "1. Continue\n";
        std::cout << "0. Exit\n";
        std::cout << "Option: ";
        std::cin >> option;
    } while (option != 0);
}

```

This is useful for simple console menus.

5.8 break and continue

`break` exits a loop:

```
for (int i = 1; i <= 10; ++i) {
    if (i == 5) {
        break;
    }
    std::cout << i << std::endl;
}
```

`continue` skips the rest of the current iteration:

```
for (int i = 1; i <= 10; ++i) {
    if (i % 2 == 0) {
        continue;
    }
    std::cout << i << std::endl;
}
```

5.9 Common Mistakes

- writing `=` instead of `==` in a condition,
- adding an accidental semicolon after `if` or `while`,
- forgetting `break` in `switch`,
- creating an infinite loop because the loop variable never changes,
- dividing by zero when no valid input was provided,
- using `for` for input that should end based on user command.

5.10 Bridge Exercise: Minimum, Maximum, and Average

Write a console program that reads real numbers from the user and calculates:

- minimum value,
- maximum value,
- arithmetic average,
- number of valid values.

The user should be able to finish input with a portable text command, for example `q`, `end`, or an empty line.

Example session:

```
Enter a number or q to finish: 10
Enter a number or q to finish: -2.5
Enter a number or q to finish: 4.5
Enter a number or q to finish: q
```

```
Count: 3
Minimum: -2.5
Maximum: 10
Average: 4
```

5.10.1 Requirements

The program should:

- use C++17,
- use a `while` or `do while` loop,
- store the sum in a `double`,
- count valid values,

- update minimum and maximum after each valid value,
- detect invalid input,
- avoid division by zero when no numbers were entered,
- compile with `-Wall -Wextra -pedantic`.

5.10.2 Suggested Work Plan

1. Write a loop that reads text until the user enters `q`.
2. Add a counter for valid numbers.
3. Add sum and average calculation.
4. Add minimum and maximum.
5. Add a message for the empty data case.
6. Add invalid input handling, for example for `abc`.

5.10.3 Check Scenarios

1. Enter 10, -2.5, 4.5, then `q`.
2. Enter only one number and check that minimum, maximum, and average are the same.
3. Finish without entering any number and check the message.
4. Enter invalid text such as `abc` and check that the program asks again.
5. Enter several negative numbers and verify minimum and maximum.

5.11 Practice Tasks

1. Read a number and print whether it is positive, negative, or zero.
2. Read a menu option and handle it with `switch`.
3. Print numbers from 1 to 100 with a `for` loop.
4. Sum numbers from 1 to `n`.
5. Read numbers until the user enters 0.
6. Print a multiplication table.
7. Skip even numbers using `continue`.
8. Stop searching when a target value is found using `break`.

5.12 Student Checklist

Before moving to functions, arrays, and strings, make sure you can:

- write an `if/else` condition,
- use comparison and logical operators,
- use a `switch` with `break`,
- choose between `for`, `while`, and `do while`,
- write a loop that depends on user input,
- explain when `break` and `continue` are useful,
- solve the minimum, maximum, and average bridge exercise.

Chapter 6

03 - Functions, Arrays, and Strings

This chapter teaches how to split a program into smaller parts and how to work with simple collections of data. You will use functions, one-dimensional arrays, two-dimensional arrays, and `std::string`.

6.1 Learning Goals

After this chapter you should be able to:

- declare and define your own functions,
- pass arguments to functions,
- return values from functions,
- understand the difference between a declaration and a definition,
- use one-dimensional arrays,
- use simple two-dimensional arrays,
- iterate over array elements,
- perform basic operations on `std::string`,
- split a larger task into smaller functions.

6.2 Why Functions Matter

A function gives a name to a piece of logic. Instead of writing everything in `main`, you can move a calculation into a separate function:

```
#include <iostream>

int add(int a, int b) {
    return a + b;
}

int main() {
    std::cout << add(2, 3) << std::endl;
}
```

This makes code easier to read, test, and reuse.

6.3 Declaration and Definition

A declaration tells the compiler that a function exists:

```
int add(int a, int b);
```

A definition contains the function body:

```
int add(int a, int b) {
    return a + b;
}
```

In small programs, both can be in one file. In larger projects, declarations often go into header files and definitions go into `.cpp` files.

6.4 Parameters and Return Values

Parameters are input values for a function. `return` sends a result back:

```
double rectangleArea(double width, double height) {
    return width * height;
}
```

Use clear names. A function named `rectangleArea` is easier to understand than one named `calc`.

6.5 Local Variables and Scope

Variables declared inside a function are local to that function:

```
int square(int value) {
    int result = value * value;
    return result;
}
```

`result` cannot be used outside `square`. This is useful because it prevents unrelated parts of the program from accidentally changing each other.

6.6 One-Dimensional Arrays

An array stores several values of the same type:

```
#include <iostream>

int main() {
    int numbers[5] = {4, 2, 9, 1, 7};

    for (int i = 0; i < 5; ++i) {
        std::cout << numbers[i] << std::endl;
    }
}
```

Array indexes start at 0. For an array of size 5, valid indexes are 0, 1, 2, 3, and 4.

6.7 Array Minimum and Maximum

```
#include <iostream>

int main() {
    int numbers[5] = {4, 2, 9, 1, 7};
    int minimum = numbers[0];
    int maximum = numbers[0];

    for (int i = 1; i < 5; ++i) {
        if (numbers[i] < minimum) {
            minimum = numbers[i];
        }
        if (numbers[i] > maximum) {
```

```

        maximum = numbers[i];
    }
}

std::cout << "Minimum: " << minimum << std::endl;
std::cout << "Maximum: " << maximum << std::endl;
}

```

The first element is a practical initial value when the array is not empty.

6.8 Two-Dimensional Arrays

A two-dimensional array can represent a table or a small matrix:

```

#include <iostream>

int main() {
    int matrix[2][3] = {
        {1, 2, 3},
        {4, 5, 6}
    };

    for (int row = 0; row < 2; ++row) {
        for (int col = 0; col < 3; ++col) {
            std::cout << matrix[row][col] << " ";
        }
        std::cout << std::endl;
    }
}

```

Nested loops are the usual way to process rows and columns.

6.9 Strings

`std::string` stores text:

```

#include <iostream>
#include <string>

int main() {
    std::string text = "C++";

    std::cout << text << std::endl;
    std::cout << "Length: " << text.size() << std::endl;
}

```

You can iterate over characters:

```

for (char ch : text) {
    std::cout << ch << std::endl;
}

```

6.10 Reading a Full Line

Use `std::getline` when the text may contain spaces:

```

#include <iostream>
#include <string>

int main() {

```

```

    std::string sentence;

    std::cout << "Enter text: ";
    std::getline(std::cin, sentence);

    std::cout << "You entered: " << sentence << std::endl;
}

```

6.11 Bridge Exercise: GADERYPOLUKI Cipher

Write a console program that encrypts text with the GADERYPOLUKI substitution cipher.

The cipher uses replacement pairs:

GA DE RY PO LU KI

Rules:

- G becomes A,
- A becomes G,
- D becomes E,
- E becomes D,
- the same rule applies to every pair,
- characters not present in the key stay unchanged.

Example:

PROGRAM -> OYPAYGM

6.12 Required Function

The program should contain a function:

```
std::string encryptGaderypoluki(const std::string& text);
```

The function should:

- take the input text as `std::string`,
- return a new encrypted `std::string`,
- not modify the input text directly,
- keep characters outside the key unchanged.

6.13 Suggested Work Plan

1. Write a helper function that replaces one character.
2. Test the pair G and A.
3. Add all remaining key pairs.
4. Write a function that loops over the whole string.
5. Read input with `std::getline`.
6. Test text containing spaces, digits, and punctuation.

6.14 Check Scenarios

1. Input PROGRAM; expected result: OYPAYGM.
2. Input GADERYPOLUKI; every letter should be replaced by its pair.
3. Input ALA 123; digits and spaces should stay unchanged.
4. Input an empty string; the program should not crash.

6.15 Extra Tasks

After the basic version works, add:

- lowercase support,
- preserving letter case,
- automatic decryption with the same function,
- tests for the encryption function,
- reading text from a file,
- writing encrypted text to a file,
- a custom substitution key.

6.16 Common Mistakes

- printing from the encryption function instead of returning a value,
- changing only one pair and forgetting the rest of the key,
- using `std::cin >> text` when the text may contain spaces,
- modifying the original string when a returned result would be clearer,
- mixing input/output code with encryption logic.

6.17 Practice Tasks

1. Write a function `isEven`.
2. Write a function `maxOfTwo`.
3. Calculate the average of an array.
4. Find the minimum and maximum in an array.
5. Print a two-dimensional array as a table.
6. Count letters in a string.
7. Check whether a string is a palindrome.
8. Implement the GADERYPOLUKI cipher.

6.18 Student Checklist

Before moving to pointers, references, and memory, make sure you can:

- write a function with parameters and a return value,
- explain local variable scope,
- iterate over an array by index,
- process a two-dimensional array with nested loops,
- read a full line with `std::getline`,
- iterate over characters in `std::string`,
- split a string-processing task into small functions.